

CURSO ACADÉMICO
2025-2026

TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

Departamento de Informática

PROYECTO FIN DE GRADO



Hospifood Quality

Manual Técnico

Autor:

RONIC ALEXANDER LEAL PÉREZ

Índice

1. Introducción	2
2. Arquitectura de la aplicación	3
2.1. Frontend	5
2.1.1. Tecnologías usadas	5
2.1.2. Justificación de la elección.....	6
2.1.3. Entorno de desarrollo.....	7
2.2. Backend	8
2.2.1. Tecnologías usadas	8
2.2.2. Justificacion	9
2.2.3. Entorno de desarrollo.....	9
3. Documentación Técnica	11
3.1. Análisis	11
3.1.1. Roles principales	11
3.1.2. Requisitos funcionales.....	12
3.1.3. Requisitos no funcionales	13
3.2. Desarrollo	14
3.2.1. Base de Datos	14
3.2.2. Justificación del diseño	16
3.2.3. Diagrama de casos de uso.....	17
3.3. Pruebas realizadas.....	18
4. Propuestas de mejoras	20
4.1. Mejoras Funcionales.....	20
5. Bibliografía.....	21

1. Introducción

El presente Manual Técnico detalla la arquitectura, el análisis, el diseño y los detalles de implementación de **Hospifood Quality**, una plataforma web desarrollada como Proyecto Final del Grado Superior en Desarrollo de Aplicaciones Web.

El propósito de la aplicación es digitalizar y optimizar el proceso de evaluación de la calidad alimentaria por parte del paciente en los hospitales públicos de Extremadura mediante la implementación del Modelo Normalizado de Calidad e Inocuidad de los Alimentos en los Hospitales Públicos de Extremadura (Método COCINHEX). Tradicionalmente ejecutado en papel, este proyecto transforma el flujo de trabajo en un sistema digital bidireccional y eficiente:

- Por un lado, proporciona a los pacientes una **interfaz táctil accesible** (vía código QR) para evaluar su satisfacción con las diferentes ingestas durante su estancia hospitalaria.
- Por otro lado, ofrece a los responsables de calidad y a la dirección un **panel de control analítico en tiempo real**.

A nivel de ingeniería de software, el reto principal ha consistido en diseñar un sistema que garantice el acceso directo y anónimo para el paciente, manteniendo al mismo tiempo una estricta seguridad, envío de alertas tempranas y aislamiento de datos (según la LOPD) para los paneles de administración de cada hospital.

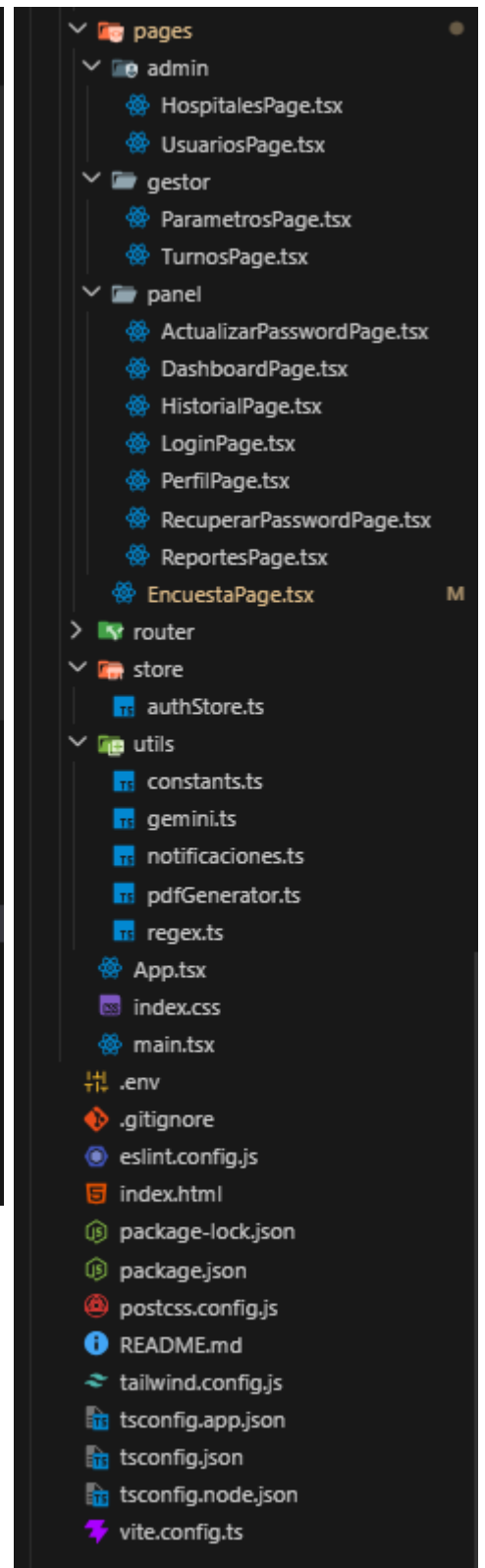
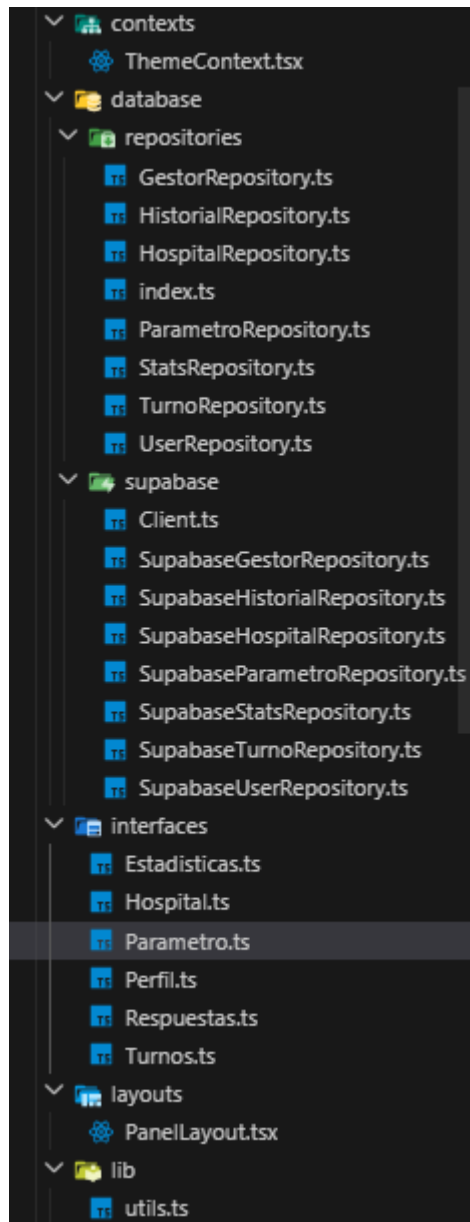
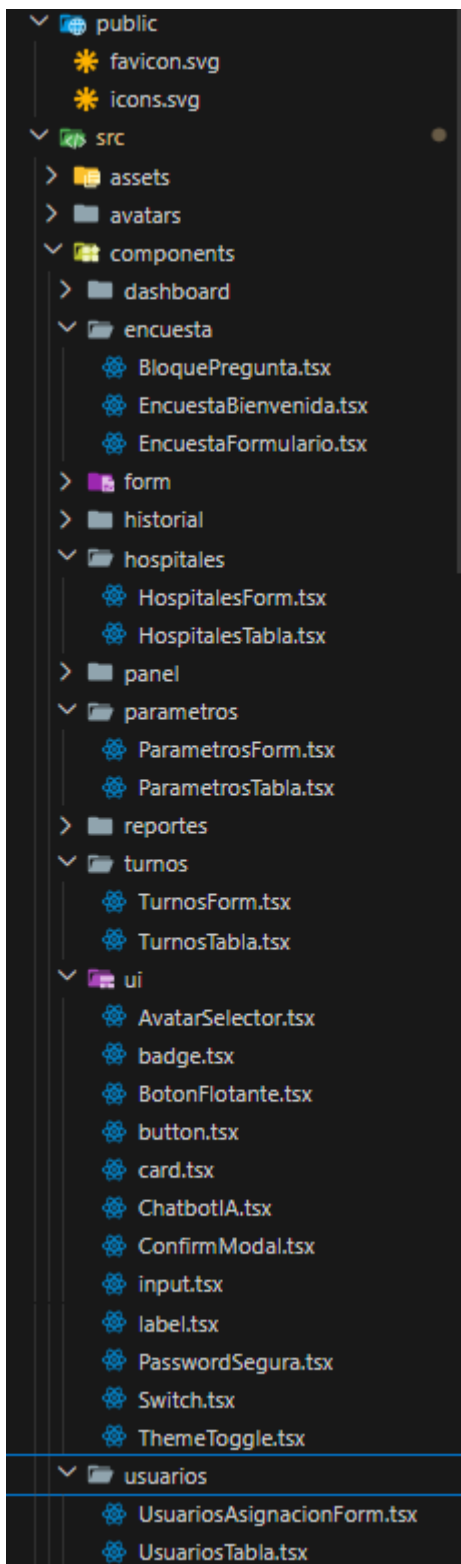
2. Arquitectura de la aplicación

La arquitectura de **Hospifood Quality** está basada en una *Single Page Application* (SPA) desarrollada con **React**. El proyecto se organiza en las siguientes capas:

- **Capa de presentación (Frontend):** componentes React, enrutado, estilos con TailwindCSS y gestión de estado global mediante Zustand.
- **Capa de servicios (BaaS):** Supabase proporciona base de datos PostgreSQL, autenticación de usuarios, almacenamiento de ficheros y políticas de seguridad a nivel de fila (*Row Level Security*).
- **Servicios externos:**
 - **EmailJS:** para el envío automatizado de notificaciones de alertas de temperatura crítica.
 - **Gemini API (Google AI):** integración de un ChatBot inteligente para la asistencia en la interpretación de datos de calidad y soporte al gestor.

La comunicación entre la aplicación y Supabase se realiza mediante la librería oficial `@supabase/supabase-js`, utilizando peticiones HTTP sobre HTTPS y respuestas en formato JSON.

En las siguientes imágenes encontramos la estructura del proyecto:



A continuación, se detalla la arquitectura diferenciando la parte de frontend y el backend (Supabase).

2.1. Frontend

La capa de presentación es una SPA que renderiza las vistas dinámicamente en el cliente, eliminando recargas de página y ofreciendo una experiencia fluida.

2.1.1. Tecnologías usadas

Para la construcción del frontend se ha seleccionado un conjunto de herramientas modernas que garantizan un rendimiento óptimo y una alta escalabilidad:

- **React (v18+)**: Librería principal basada en componentes funcionales y *hooks* para la creación de interfaces de usuario dinámicas.
- **Vite**: Herramienta de *bundling* que proporciona una velocidad superior en el desarrollo y una compilación optimizada para producción.
- **Tailwind CSS (v4.0 con @tailwindcss/vite)**: Motor de estilos de última generación basado en utilidades, configurado mediante el nuevo plugin nativo para Vite, permitiendo un diseño *responsive* y soporte nativo de modo oscuro con mayor rendimiento.
- **Shadcn/UI**: Colección de componentes de interfaz altamente accesibles y personalizables, contruidos sobre Radix UI y Tailwind, utilizados para garantizar una estética profesional y consistente en toda la plataforma.
- **Zustand**: Gestor de estado global ligero utilizado para la persistencia de la sesión de usuario, gestión de perfiles y sincronización de preferencias de notificaciones.
- **React Router DOM**: Sistema de enrutado en el cliente para la gestión de navegación entre las vistas públicas (encuestas) y las rutas protegidas (panel de gestión).
- **Recharts**: Librería de visualización de datos basada en componentes de React y SVG, empleada para representar gráficamente las métricas de calidad alimentaria (sabor, temperatura, higiene, etc.).
- **jsPDF & html2canvas**: Conjunto de librerías para la generación de documentos PDF directamente desde el navegador, permitiendo al gestor exportar los reportes visuales y estadísticas de calidad de forma inmediata.

- **EmailJS:** Servicio utilizado para el envío automatizado de correos electrónicos transaccionales (tanto para alertas de temperatura crítica como para el envío de reportes de calidad) sin necesidad de infraestructura de servidor propia.
- **Google Generative AI SDK (Gemini Pro):** Integración del modelo de inteligencia artificial de Google para dotar a la aplicación de un ChatBot analítico capaz de asistir al gestor en la interpretación de los datos recopilados.
- **Lucide React:** Biblioteca de iconos vectoriales optimizada para mejorar la usabilidad visual en menús, botones y escalas de valoración.

Esta lista cubre todo el espectro de herramientas que dan vida a la interfaz, desde el diseño (Shadcn/Tailwind) hasta las funciones complejas de análisis (Gemini) y salida de datos (PDF/EmailJS).

2.1.2. Justificación de la elección

La selección del stack tecnológico para Hospifood Quality no ha sido arbitraria, sino que responde a criterios de agilidad, accesibilidad y eficiencia operativa:

- **React y Vite:** Se han elegido para garantizar un desarrollo ágil mediante una arquitectura de componentes modulares reutilizables, como los selectores de plantas, turnos y el flujo de preguntas secuenciales. Vite proporciona una velocidad de respuesta instantánea durante el desarrollo y optimiza el empaquetado final, asegurando que la aplicación cargue con rapidez desde dispositivos móviles y red Wi-Fi.
- **Tailwind CSS (v4.0) y Shadcn/UI:** Esta combinación ha permitido implementar de forma eficiente el diseño ***Fat Finger***, caracterizado por botones de gran tamaño y elementos táctiles claros, esenciales para la accesibilidad de los pacientes que utilizan tablets y móviles en sus habitaciones. Al utilizar Shadcn/UI, se garantiza una interfaz con estándares profesionales de accesibilidad (aria-attributes) y una estética limpia sin el sobrecoste de mantener archivos CSS voluminosos.
- **Zustand:** Frente a alternativas más complejas como Redux, Zustand ofrece una gestión de estado global extremadamente ligera. Se ha

justificado su uso para centralizar la sesión del personal sanitario y asegurar que las preferencias de notificaciones y el perfil del usuario se mantengan persistentes de forma sencilla entre recargas de página.

- **Recharts y Gemini API:** La interpretación de los datos es un pilar del método COCINHEX. Mientras que **Recharts** permite al responsable de calidad visualizar de un vistazo las medias de satisfacción mediante gráficos vectoriales nítidos, la integración de **Gemini** añade una capa de inteligencia artificial analítica capaz de resumir tendencias complejas y ofrecer soporte interpretativo inmediato al responsable de calidad.
- **jsPDF, html2canvas y EmailJS:** Estas herramientas permiten mantener la arquitectura **serverless** (sin servidor propio). **jsPDF** y **html2canvas** facultan la creación de reportes profesionales en formato PDF procesados íntegramente en el navegador del cliente. Por su parte, **EmailJS** garantiza que tanto las alertas de seguridad alimentaria (temperatura crítica) como el envío de informes lleguen a los destinatarios sin necesidad de configurar y mantener un servidor de correo SMTP propio.
- **TypeScript:** Su implementación es fundamental para dotar al proyecto de robustez, detectando errores de tipado en tiempo de desarrollo y facilitando el mantenimiento a largo plazo del código fuente, especialmente en la comunicación con los servicios de Supabase.

2.1.3. Entorno de desarrollo

- **Sistema operativo:** Windows 11.
- **IDE:** Visual Studio Code, configurado con extensiones críticas para el desarrollo: TypeScript, React, Tailwind CSS IntelliSense, ESLint y Prettier para garantizar la calidad y formato del código.
- **Gestor de dependencias:** npm (incluido con Node.js v18+), utilizado para la administración de las librerías del proyecto.
- **Navegador para pruebas:** Google Chrome, utilizando las *Chrome DevTools* de manera intensiva para la depuración del estado de React, la simulación de resoluciones táctiles (móviles) y la supervisión del almacenamiento en *LocalStorage* para el control de encuestas duplicadas.

- **Control de versiones:** Git + GitHub, donde se almacena el repositorio con el código fuente, permitiendo un control de versiones riguroso y la integración con el sistema de despliegue.

Este entorno ha sido seleccionado por ser el estándar predominante en el desarrollo web moderno, ofrecer una integración nativa y robusta con TypeScript y permitir un flujo de trabajo ágil y automatizado con la plataforma de despliegue Vercel.

2.2. Backend

En este proyecto no se ha desarrollado un backend propio con un framework tradicional (como Spring Boot o Express.js). En su lugar, se ha optado por un enfoque Backend as a Service (BaaS) utilizando Supabase, lo que permite delegar la infraestructura y centrar el desarrollo en la lógica de negocio y la interfaz del usuario.

2.2.1. Tecnologías usadas

- **Supabase:**
 - **PostgreSQL:** Base de datos relacional utilizada para almacenar la estructura completa del método COCINHEX: hospitales, perfiles de personal, parámetros de calidad, turnos, encuestas y sus respectivas respuestas.
 - **Supabase Auth:** Sistema encargado de la autenticación de los gestores y administradores, manejando de forma segura el inicio de sesión y el control de sesiones mediante JSON Web Tokens (JWT).
 - **Supabase Storage:** Utilizado para el almacenamiento y servicio de las imágenes de los avatares personalizados para los perfiles de los responsables de calidad.
 - **Row Level Security (RLS):** Pilar de la seguridad del proyecto que permite definir políticas a nivel de fila en PostgreSQL para garantizar que cada responsable solo acceda a los datos de los hospitales que tiene asignados, asegurando el aislamiento de datos entre centros.

- **Librería de acceso:**
 - **@supabase/supabase-js:** Cliente oficial utilizado para realizar las operaciones CRUD (crear, leer, actualizar, borrar) y la gestión de la autenticación directamente desde el frontend de React.
- **Servicios externos:**
 - **EmailJS:** Integración para la gestión de envíos de correos electrónicos automatizados (alertas de temperatura y reportes de calidad) directamente desde el cliente.
 - **Gemini API (Google AI):** Integración de inteligencia artificial para el análisis de datos de satisfacción y la asistencia interactiva al gestor a través del ChatBot integrado.

2.2.2. Justificación

- **Supabase** proporciona en una única plataforma todas las herramientas necesarias (Base de datos, Autenticación y Almacenamiento), lo que ha reducido drásticamente los tiempos de desarrollo y la complejidad operativa del proyecto.
- Al utilizar **PostgreSQL**, se cuenta con un motor de base de datos de nivel empresarial, potente y estándar, que permite realizar consultas complejas para los reportes estadísticos y gestionar la integridad referencial de manera robusta.
- La arquitectura **Serverless** resultante elimina la necesidad de mantenimiento de servidores, parches de seguridad y administración de sistemas, lo que es ideal para una implementación eficiente en el entorno sanitario.

2.2.3. Entorno de desarrollo

- **Dashboard web de Supabase:** Para la creación y modelado visual de las tablas, gestión de relaciones y administración de los registros en tiempo real.
- **Editor SQL de Supabase:** Para la ejecución de scripts DDL y la configuración precisa de las políticas RLS de seguridad.

- **Vercel:** Plataforma de despliegue del frontend, configurada con las variables de entorno necesarias para la conexión segura con Supabase en producción. La aplicación se encuentra desplegada en:
<https://hospifood-quality.vercel.app/>

3. Documentación Técnica

3.1. Análisis

El análisis de este proyecto se centra en la transformación del método de evaluación de la satisfacción del paciente en una herramienta digital de alta precisión. El sistema ha sido diseñado para equilibrar la facilidad de uso para el paciente con la profundidad analítica requerida por el Responsable de Calidad (Veterinario Bromatólogo).

3.1.1. Roles principales

Se han definido tres niveles de acceso, cada uno con permisos y responsabilidades claramente diferenciados mediante políticas de seguridad:

- **Paciente (Actor Público):** Es el usuario que realiza la valoración. No requiere autenticación para eliminar fricciones en la toma de datos. Accede mediante un código QR y su interacción es totalmente anónima.
- **Responsable de Calidad (Actor Privado):** Personal del hospital con acceso autenticado. Es el responsable de monitorizar los resultados en tiempo real, recibir alertas de seguridad alimentaria, exportar reportes y consultar al asistente de IA (Gemini). Sus permisos están restringidos únicamente a los hospitales que tiene asignados.
- **Administrador (Superusuario):** Tiene control total sobre la plataforma. Sus funciones incluyen la visión global de todas las encuestas filtradas por plantas, turnos configurados y parámetros evaluados, gestión de hospitales (CRUD), asignación de hospitales para los responsables de calidad y eliminación de sus cuentas.

3.1.2. Requisitos funcionales

Los requisitos funcionales describen las acciones específicas que el sistema permite realizar:

- **RF1. Realización de encuestas:** El sistema debe permitir al paciente completar una valoración secuencial (Planta, Turno, Parámetros y Sugerencias) sin necesidad de registro.
- **RF2. Control de duplicidad:** El sistema debe implementar un mecanismo (LocalStorage) para evitar el spam, impidiendo que un mismo dispositivo envíe más de una encuesta cada 4 horas.
- **RF3. Dashboard analítico:** Los responsables de calidad (gestores) deben visualizar gráficos interactivos (Recharts) que muestren las medias de satisfacción por valoraciones y turno.
- **RF4. Alertas de seguridad alimentaria:** El sistema debe enviar automáticamente un correo electrónico (EmailJS) a los responsables si la valoración de "Temperatura" es crítica (valor ≤ 2).
- **RF5. Generación de informes:** El responsable de calidad debe poder exportar las estadísticas actuales en formato PDF directamente desde la interfaz y poder enviar los informes por correo electrónico.
- **RF6. Asistencia por IA:** La aplicación debe integrar un ChatBot (Gemini Pro) para el responsable de calidad capaz de interpretar los datos del hospital y responder preguntas sobre tendencias de calidad.
- **RF7. Gestión de perfil:** Los usuarios autenticados deben poder actualizar sus credenciales de acceso, avatar y preferencias de notificación.
- **RF8. Gestion Administrador:** El administrador puede tener una visión global del resultado de las encuestas en todos los hospitales, visualizando como esta configurado cada hospital, responsable asignado, los turnos y parámetros evaluados. Puede dar de alta a hospitales, eliminar y activar o desactivar. Puede asignar responsables de calidad a un hospital y eliminar sus cuentas.

3.1.3. Requisitos no funcionales

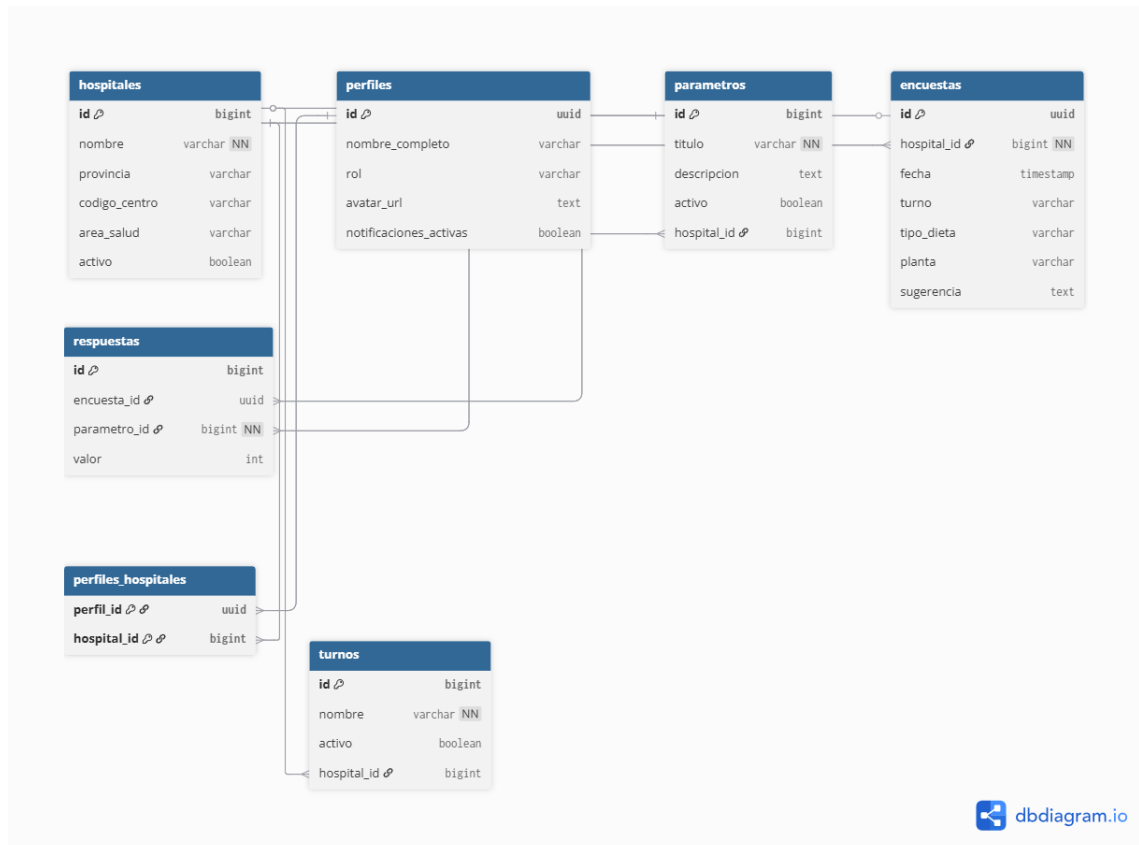
Estos requisitos definen las cualidades y restricciones técnicas del sistema:

- **RNF1. Seguridad y Aislamiento (RLS):** El sistema debe garantizar que los datos de un hospital sean invisibles para los gestores de otros centros mediante políticas de *Row Level Security* en la base de datos.
- **RNF2. Rendimiento (SPA):** La navegación entre secciones debe ser instantánea y sin recargas de página para ofrecer una experiencia fluida.
- **RNF3. Accesibilidad (Fat Finger Design):** La interfaz de la encuesta debe tener elementos táctiles de gran tamaño, optimizados para su uso en móviles y tablets por personas con movilidad reducida.
- **RNF4. Disponibilidad y Escalabilidad:** Al ser una arquitectura *front-only* sobre infraestructura BaaS, el sistema debe ser capaz de soportar picos de tráfico durante los turnos de comida sin degradación del servicio.
- **RNF5. Privacidad:** Se debe garantizar el anonimato absoluto de los pacientes, no almacenando ningún dato de carácter personal que permita su identificación.

3.2. Desarrollo

3.2.1. Base de Datos

La persistencia de datos de Hospifood Quality se apoya en un modelo relacional gestionado por PostgreSQL en Supabase. El diseño permite el aislamiento de datos por centro y el cumplimiento del método COCINHEX mediante las siguientes entidades:

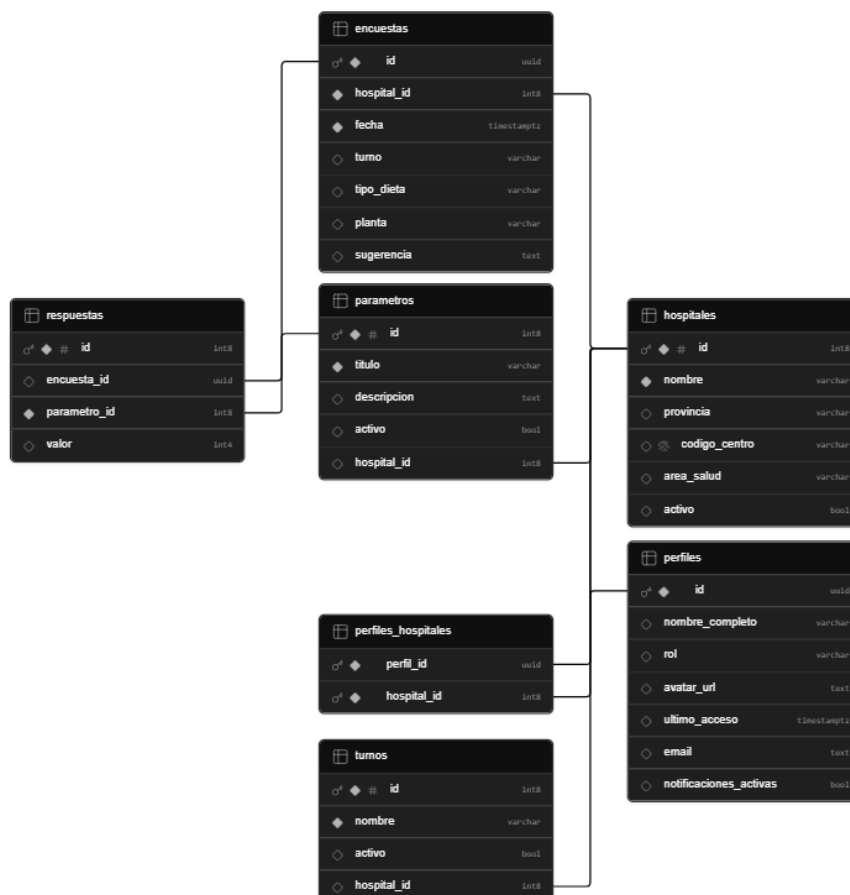


- **hospitales**: Almacena la información de cada centro (código, área de salud, provincia) y su estado (activo/inactivo).
- **perfiles**: Contiene los datos del personal (nombre, rol, avatar) y está vinculada directamente al esquema de autenticación (auth.users) mediante el campo id (UUID).
- **perfiles_hospitales**: Tabla intermedia que gestiona la relación N:M entre gestores y centros, permitiendo que un responsable supervise varios hospitales.

- **turnos y parametros:** Definen los horarios de comida y los indicadores COCINHEX (Sabor, Temperatura, etc.) configurables por cada hospital.
- **encuestas:** Registra la cabecera de cada valoración (planta, tipo de dieta, turno). Nótese que **no existe relación con la tabla perfiles** para garantizar el anonimato total del paciente.
- **respuestas:** Almacena la puntuación numérica para cada parámetro de una encuesta específica.

La estructura de tablas se ha diseñado teniendo en cuenta la integración directa con el sistema de autenticación Supabase Auth, lo cual influye en la forma en que se presentan las relaciones dentro del modelo.

A continuación, se muestra la representación generada por el propio panel de Supabase:



3.2.2. Justificación del diseño

El diseño del esquema relacional no solo busca almacenar información, sino responder de forma estratégica a las exigencias de un entorno crítico como es el hospitalario. Los tres pilares de esta justificación son:

1. Escalabilidad Multi-hospital (Multi-tenancy):

- El esquema sigue un modelo de arquitectura "multi-inquilino" lógico. Gracias a la inclusión del campo `hospital_id` como clave foránea en las tablas de configuración (turnos, parámetros) y de datos (encuestas), el sistema permite la convivencia de múltiples centros sanitarios en una única base de datos.
- Esto facilita la gestión centralizada por parte del Administrador, quien puede añadir nuevos hospitales de forma instantánea sin necesidad de realizar cambios en el código o desplegar nuevas instancias de la base de datos.

2. Seguridad y Privacidad (Anonimato por Diseño):

- Siguiendo los principios de la LOPD y el Reglamento General de Protección de Datos (RGPD) en el ámbito sanitario, se ha aplicado un desacoplamiento total entre los sujetos y sus opiniones.
- A diferencia de otras plataformas, la entidad encuestas carece intencionadamente de cualquier relación con identificadores de usuario o pacientes. Esta estructura garantiza que, incluso en caso de una auditoría técnica, sea imposible rastrear o vincular una respuesta específica con un paciente concreto, blindando el anonimato de la valoración.

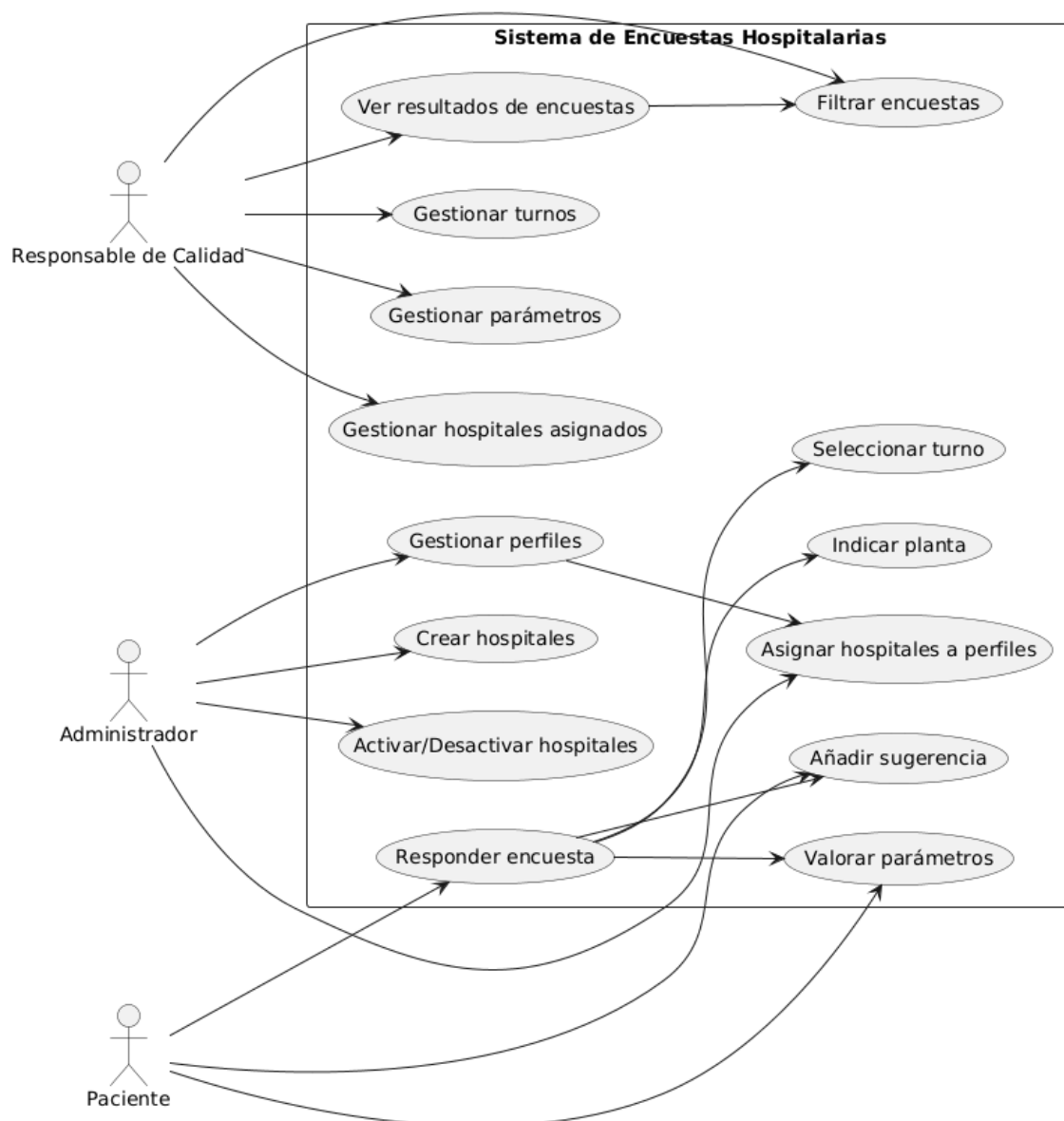
3. Integridad Referencial y Trazabilidad Histórica:

- El uso de claves foráneas (FK) asegura la consistencia de los datos en todo el ciclo de vida de la información. Por ejemplo, una respuesta no puede quedar "huérfana", ya que depende obligatoriamente de un `parametro_id` y un `encuesta_id` válidos.
- Para proteger la integridad de las estadísticas, se ha optado por un sistema de **desactivación lógica** mediante el campo activo en lugar del borrado físico de registros. Esto permite que, si un hospital deja de utilizar

un parámetro COCINHEX o un turno, los datos históricos de las encuestas pasadas permanezcan intactos y coherentes para análisis comparativos anuales.

3.2.3. Diagrama de casos de uso

El siguiente diagrama detalla las interacciones entre los actores y el sistema de encuestas hospitalarias:



El sistema diferencia claramente las responsabilidades de cada actor:

- **Paciente:** Su interacción se limita al nodo **Responder encuesta**, que engloba seleccionar dieta, turno, planta, valorar los parámetros COCINHEX y añadir sugerencias de texto.
- **Responsable de Calidad:** Interactúa con el sistema para la explotación de datos (**Ver resultados**, **Filtrar encuestas**) y la gestión operativa de su centro (**Gestionar turnos y parámetros**).
- **Administrador:** Actúa sobre la infraestructura global del sistema, encargándose de **Gestionar perfiles**, **Crear hospitales** y realizar la **Asignación de centros** a los responsables de calidad.

3.3. Pruebas realizadas

Aunque no se ha implantado un framework de pruebas automatizadas completo, se han llevado a cabo pruebas manuales sistemáticas sobre las funcionalidades principales:

→ Pruebas de Autenticación y Control de Acceso

- **Datos de entrada:** Credenciales válidas e inválidas; intentos de acceso directo a URLs protegidas (ej. /dashboard).
- **Resultado esperado:** El sistema permite el acceso solo con credenciales correctas; el enrutamiento protegido redirige al usuario no autenticado a la pantalla de login.

→ Pruebas de Control de Duplicidad (Antispam)

- **Datos de entrada:** Intento de realizar una segunda encuesta desde el mismo navegador en un intervalo inferior a 4 horas.
- **Resultado esperado:** El sistema detecta la marca de tiempo en el LocalStorage y muestra la vista informativa de "Opinión ya recibida", impidiendo la saturación de la base de datos.

→ **Pruebas de Alertas de Seguridad Alimentaria**

- **Datos de entrada:** Envío de una encuesta con una puntuación de "Temperatura" ≤ 2 .
- **Resultado esperado:** Activación inmediata del servicio **EmailJS** y recepción del correo de alerta en la bandeja de entrada del gestor asignado.

→ **Pruebas de Aislamiento de Datos (RLS)**

- **Datos de entrada:** Inicio de sesión con dos gestores de diferentes hospitales.
- **Resultado esperado:** Gracias a las políticas de *Row Level Security* en Supabase, cada gestor visualiza únicamente las encuestas y estadísticas de sus centros asignados, validando la privacidad de los datos.

→ **Pruebas del Dashboard y Gráficos**

- **Datos de entrada:** Filtrado de resultados por turno (Desayuno/Comida/Cena) y tipo de dieta.
- **Resultado esperado:** Recálculo instantáneo de las medias y actualización dinámica de los gráficos de Recharts sin latencia perceptible.

→ **Pruebas de Asistencia por IA (Gemini)**

- **Datos de entrada:** Preguntas formuladas al ChatBot sobre tendencias de satisfacción o incidencias recurrentes.
- **Resultado esperado:** Respuesta coherente y analítica generada por el modelo Gemini Pro, demostrando una correcta integración del SDK de Google.

→ **Pruebas de Generación de Reportes PDF**

- **Datos de entrada:** Solicitud de descarga de informe desde el panel de gestión.
- **Resultado esperado:** Generación correcta del archivo mediante jsPDF y html2canvas, manteniendo el diseño visual del Dashboard en el documento final.

4. Propuestas de mejoras

Tras la culminación de la primera fase de **Hospifood Quality**, se han identificado diversas líneas de evolución para potenciar el impacto de la herramienta en la gestión de la calidad e inocuidad de los alimentos:

4.1. Mejoras Funcionales

- **Localización automática por cama/habitación:** Integrar la aplicación con el sistema de admisiones para que, al escanear el QR, el sistema sepa no solo la planta, sino la habitación exacta. Esto permitiría al gestor identificar patrones de bandejas frías en zonas específicas del hospital.
- **Evidencia fotográfica de incidencias:** Permitir que el paciente pueda adjuntar una fotografía de la bandeja en caso de valoraciones críticas en "Presentación" o "Higiene", aportando una prueba visual inmediata al gestor de calidad.
- **Panel para personal del Servicio de Alimentación:** Crear una vista simplificada y de solo lectura para el personal de cocina, permitiéndoles ver las puntuaciones del turno anterior en tiempo real para aplicar correcciones inmediatas antes del siguiente servicio.

4.2. Mejoras Técnicas

- **Capacidad Offline (PWA):** Convertir la aplicación en una *Progressive Web App* para que los datos se guarden localmente si el paciente pierde la conexión Wi-Fi, sincronizándose con Supabase automáticamente al recuperar la señal.
- **QR Dinámicos:** Utilizar códigos QR dinámicos en pantallas de las habitaciones que caduquen cada 15 minutos.
- **Automatización de Reportes mediante Cron Jobs:** Implementar una función que genere y envíe automáticamente el PDF con el resumen semanal a la Dirección del hospital cada lunes a primera hora.

5. Bibliografía

- **Modelo Normalizado de Calidad e Inocuidad de los Alimentos en los Hospitales Públicos de Extremadura. Método COCINHEX.** Yolanda Márquez Polo, Almudena Pérez Rodríguez, Manuel del Pozo Mariño. Junta de Extremadura. Mérida, octubre de 2024.
- **React Documentation:** Guías sobre componentes funcionales, *hooks* y gestión de estado. <https://react.dev/>
- **Vite Guide:** Configuración del entorno de desarrollo y empaquetado. <https://vitejs.dev/>
- **Supabase Docs:** Documentación sobre PostgreSQL, Auth y Row Level Security. <https://supabase.com/docs>
- **Tailwind CSS (v4.0):** Documentación del motor de estilos y utilidades. <https://tailwindcss.com/>
- **Google AI Studio:** Referencia técnica para la integración de la API de Gemini Pro. <https://aistudio.google.com/>
- **EmailJS Documentation:** Gestión de correos electrónicos desde el lado del cliente. <https://www.emailjs.com/docs/>